Dokumentacja techniczna

Kalendarz Roku Szkolnego

Zespół Szkół nr 4 im. Janusza Groszkowskiego w Tychach

Wersja dokumentu: 1.0
Data: listopad 2026
Stack: Next.js · Better Auth · SQLite · Drizzle ORM · Tailwind CSS

---

1. Opis systemu

Aplikacja webowa do prezentacji i zarządzania kalendarzem roku szkolnego.

Użytkownik publiczny może:

- wyświetlić kalendarz aktywnego roku szkolnego (wrzesień–sierpień),
- zobaczyć wydarzenia przypisane do dat z kolorami kategorii,
- korzystać z legendy oznaczeń.

Administrator (po zalogowaniu) może:

- zarządzać wydarzeniami (dodawanie, edycja, usuwanie, filtrowanie),
- zarządzać latami szkolnymi (w tym ustawianie aktywnego roku),
- zarządzać kategoriami wydarzeń (nazwa, kolor, wzorzec),
- wylogować się z panelu.

Dane są przechowywane w bazie SQLite i są trwałe po restarcie aplikacji.

---

2. Wymagania funkcjonalne (realizacja)

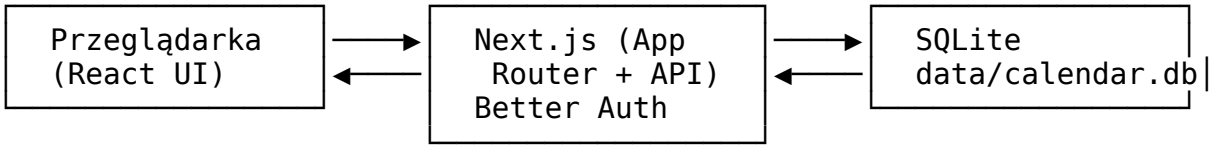
2.1. Użytkownik

Wymaganie	Realizacja
Kalendarz aktywnego roku szkolnego	Strona główna `/` – dane z bazy
Wszystkie miesiące w czytelnym układzie	Komponenty `CalendarView`, `MonthCalendar`
Wydarzenia na konkretnych datach	Kolorowanie dni według kategorii
Kategorie kolorami + legenda	`Legend` + kolory z tabeli `categories`
Dni wolne, egzaminy, zebrania itd.	Seed + CRUD administratora
Zmiana roku – dane z bazy	Aktywny rok (`is_active`)

2.2. Administrator

Wymaganie	Realizacja
Logowanie / wylogowanie	Better Auth, `/admin/login`
Lista wydarzeń + filtry	`/admin/events`
CRUD wydarzeń	API `/api/events`
Kategorie i lata szkolne	`/admin/categories`, `/admin/years`
Aktywny rok szkolny	Checkbox przy edycji roku
Walidacja formularzy	Kontrola wymaganych pól (klient + API)
Zmiany tylko po zalogowaniu	Sprawdzanie sesji i `role === "admin"`
Zapis w bazie	SQLite przez Drizzle ORM

3. Architektura



- **Frontend:** React (komponenty klienckie `"use client"` oraz strony serwerowe).
- **Backend:** Route Handlers Next.js (`src/app/api/...`).
- **Auth:** Better Auth (sesja w ciasteczkach HTTP-only).
- **Baza:** jeden plik SQLite, dostęp synchroniczny przez `better-sqlite3` + Drizzle.

---

4. Stack technologiczny

Warstwa	Technologia
Framework	Next.js 16 (App Router)
Język	TypeScript
UI	React 19, Tailwind CSS v4
Autentykacja	Better Auth (e-mail + hasło)
ORM	Drizzle ORM
Baza	SQLite (`better-sqlite3`), plik `data/calendar.db`
Narzędzia	tsx, drizzle-kit

5. Struktura projektu

```
calendar/
├── data/
│   └── calendar.db                # baza SQLite
├── public/                        # pliki statyczne
├── scripts/
│   └── create-admin.ts            # nadanie roli admin
├── src/
│   ├── app/
│   │   ├── page.tsx               # kalendarz publiczny
│   │   ├── layout.tsx
│   │   ├── globals.css
│   │   ├── admin/
│   │   │   ├── page.tsx           # dashboard
│   │   │   ├── login/page.tsx
│   │   │   ├── events/page.tsx
│   │   │   ├── years/page.tsx
│   │   │   └── categories/page.tsx
│   │   └── api/
│   │       ├── auth/[...all]/route.ts
│   │       ├── events/
│   │       │   ├── route.ts              # GET, POST
│   │       │   └── [id]/route.ts         # PUT, DELETE
│   │       ├── school-years/
│   │       │   ├── route.ts              # GET, POST
│   │       │   └── [id]/
│   │       │       ├── route.ts          # PUT, DELETE
│   │       │       └── activate/route.ts
│   │       └── categories/
│   │           ├── route.ts              # GET, POST
│   │           └── [id]/route.ts         # PUT, DELETE
│   ├── components/
│   │   ├── CalendarView.tsx
│   │   ├── MonthCalendar.tsx
│   │   ├── Legend.tsx
│   │   └── ConfirmModal.tsx
│   ├── db/
│   │   ├── index.ts               # połączenie z bazą
│   │   ├── schema.ts              # schemat tabel
│   │   └── seed.ts                # dane początkowe
│   └── lib/
│       ├── auth.ts                # Better Auth (serwer)
│       ├── auth-client.ts         # klient auth
│       └── calendar-utils.ts
├── drizzle.config.ts
├── package.json
├── tsconfig.json
└── next.config.ts
```

---

6. Model danych

6.1. Tabele aplikacji

school_years

Kolumna	Typ	Opis
id	INTEGER PK	Identyfikator
name	TEXT UNIQUE	np. `2026/2027`
is_active	BOOLEAN	Aktywny rok (domyślnie wyświetlany)
start_date	TEXT	Data ISO `YYYY-MM-DD`
end_date	TEXT	Data ISO
created_at, updated_at	timestamp	Metadane

categories

Kolumna	Typ	Opis
id	INTEGER PK	Identyfikator
name	TEXT UNIQUE	Nazwa kategorii
color	TEXT	Kolor hex (np. `#e53935`)
description	TEXT	Opis opcjonalny
pattern	TEXT	np. `stripes`
sort_order	INTEGER	Kolejność w legendzie

events

Kolumna	Typ	Opis
id	INTEGER PK	Identyfikator
title	TEXT	Tytuł wydarzenia
description	TEXT	Opis
start_date	TEXT	Początek (ISO)
end_date	TEXT	Koniec zakresu (opcjonalnie)
category_id	FK → categories	Kategoria
school_year_id	FK → school_years	Rok szkolny
all_day	BOOLEAN	Całodniowe

6.2. Tabele Better Auth

`user`, `session`, `account`, `verification` – standardowy schemat biblioteki.
W tabeli `user` pole dodatkowe: **role** (`admin` | `user`).

---

7. Autentykacja i autoryzacja

1. Logowanie: `/admin/login` (e-mail + hasło).
2. Endpoint auth: `/api/auth/[...all]` (Better Auth).
3. Sesja przechowywana w ciasteczkach HTTP-only.
4. Endpointy modyfikujące dane (POST / PUT / DELETE) wymagają sesji z `role === "admin"`.
5. Odczyt list (GET events, categories, school-years) jest publiczny.

Utworzenie konta administratora

```
# rejestracja
curl -X POST http://localhost:3000/api/auth/sign-up/email \
  -H "Content-Type: application/json" \
  -d
"{\"email\":\"admin@school.local\",\"password\":\"admin123\",\"name\":
\"Administrator\"}"

# nadanie roli admin
npx tsx scripts/create-admin.ts admin@school.local
```

---

8. API REST

Metoda	Ścieżka	Auth	Opis
GET	`/api/events`	nie	Lista wydarzeń (`schoolYearId`, `categoryId`)
POST	`/api/events`	admin	Nowe wydarzenie
PUT	`/api/events/[id]`	admin	Edycja
DELETE	`/api/events/[id]`	admin	Usunięcie
GET	`/api/school-years`	nie	Lista lat
POST	`/api/school-years`	admin	Nowy rok
PUT	`/api/school-years/[id]`	admin	Edycja roku
DELETE	`/api/school-years/[id]`	admin	Usunięcie roku
POST	`/api/school-years/[id]/activate`	admin	Ustaw aktywny

Metoda	Ścieżka	Auth	Opis
GET	`/api/categories`	nie	Lista kategorii
POST	`/api/categories`	admin	Nowa kategoria
PUT	`/api/categories/[id]`	admin	Edycja
DELETE	`/api/categories/[id]`	admin	Usunięcie

Przykładowe ciało POST wydarzenia:

```
{
  "title": "Zebranie z rodzicami",
  "description": "g. 17:00",
  "startDate": "2026-09-08",
  "endDate": null,
  "categoryId": 5,
  "schoolYearId": 1
}
```

---

9. Interfejs użytkownika

9.1. Strona publiczna (`/`)

- Nagłówek z logo ZS4 i nazwą szkoły.
- Kolorystyka zgodna ze stroną szkoły (`#3f3b3a`, `#82000a`).
- Siatka 12 miesięcy roku szkolnego.
- Legenda kategorii i lista ważnych terminów.
- Link do panelu administratora.

9.2. Panel administratora (`/admin`)

- Dashboard ze statystykami (liczba wydarzeń, lat, kategorii).
- Sekcje: Wydarzenia, Lata szkolne, Kategorie.
- Modale potwierdzenia (`ConfirmModal`) przy usuwaniu i wylogowaniu.
- Formularze z walidacją pól wymaganych.

---

10. Uruchomienie lokalne

Wymagania

- Node.js **20 LTS** (zalecane na Windows – unika problemów z kompilacją `better-sqlite3`)
- npm

Kroki

```
# instalacja zależności
npm install
```

```
# utworzenie schematu bazy (jeśli brak pliku / tabel)
npx drizzle-kit push

# opcjonalnie — dane przykładowe roku 2026/2027
npm run db:seed

# serwer deweloperski
npm run dev
```

Aplikacja: **http://localhost:3000**

Skrypty npm

Skrypt	Opis
`npm run dev`	Serwer deweloperski
`npm run build`	Build produkcyjny
`npm run start`	Uruchomienie builda
`npm run db:push`	Synchronizacja schematu z bazą
`npm run db:seed`	Wypełnienie danymi startowymi

11. Przenoszenie projektu (np. pendrive)

Usuń przed kopiowaniem:

- `node_modules/`
- `.next/`

Zostaw m.in.:

- `src/`, `public/`, `scripts/`, `data/` (jeśli chcesz zachować dane),
- `package.json`, `package-lock.json`,
- `tsconfig.json`, `next.config.ts`, `drizzle.config.ts`.

Na drugim komputerze:

```
npm install
npm run dev
```

Przy Node 20 LTS instalacja `better-sqlite3` zwykle nie wymaga Pythona ani Visual Studio Build Tools.

---

12. Bezpieczeństwo

- Hasła hashowane przez Better Auth (nie są przechowywane jawnym tekstem).
- Operacje zapisu tylko dla sesji z rolą `admin`.
- Brak publicznego formularza rejestracji w UI (konto tworzy się przez API / skrypt).

- Baza jako plik lokalny – w środowisku produkcyjnym należy ograniczyć dostęp do systemu plików i rozważyć kopie zapasowe `data/calendar.db`.

---

13. Znane ograniczenia

1. **Windows + Node 24:** `better-sqlite3` może wymagać kompilacji (Python, narzędzia C++). Rozwiązanie: Node 20 LTS.
2. **Brak UI zarządzania użytkownikami** – role admin nadaje się skryptem / SQL.
3. **Jedna aktywna baza plikowa** – aplikacja nie jest przygotowana pod klaster wielu instancji bez współdzielonego storage.
4. **Usuwanie kategorii / roku** z powiązanymi wydarzeniami może zakończyć się błędem FK (zamierzone ograniczenie integralności).

---

14. Kolorystyka (identyfikacja wizualna)

Element	Kolor
Paski nagłówka / stopki	`#3f3b3a`
Akcenty tekstu / usuwanie	`#82000a`
Tło strony	`#f5f5f5`
Logo	zewnętrzny URL szkoły lub plik lokalny

Wzór kolorystyczny zbliżony do oficjalnej strony: https://www.zs4.oswiata.tychy.pl/

---

15. Autorstwo i kontekst

Projekt realizuje cele opisane w:

- **Projekt.txt** – responsywna aplikacja kalendarza z danymi z bazy i CRUD dla uprawnionego użytkownika,
- **Wymagania_Użytkownika.txt**,
- **Wymagania_Administratora.txt**,

oraz układ informacyjny wzorowany na graficznym kalendarzu roku szkolnego 2026/2027 ZS nr 4 w Tychach.

Koniec dokumentacji technicznej